

Algoritmos e Programação de Computadores I

Tipos, Variáveis e Constantes em C

Prof. Marcelo Farias

Objetivos da Aula

- Reconhecer os tipos primitivos do C e seus modificadores.

Declarar, inicializar e nomear variáveis e constantes.
Diferenciar conversão implícita e explícita de tipos.

Tipos Primitivos

- Tipos de dados básicos suportados pela linguagem.
- São os mais simples possível, por exemplo, um caractere, um número ou um valor lógico.
- Servem de base para criação de tipos compostos.

Tipos Primitivos no ANSI C

char	caractere
int	inteiro
float	ponto flutuante
double	ponto flutuante de precisão dupla
void	sem valor

Char

- Representa um caractere alfanumérico.
- Valor deve ser definido entre aspas simples ' .
- Caracteres especiais iniciam com barra invertida \ .
- Ocupa sempre 1 byte.

Exemplo de Valores Caracteres

'a'	letra a
"a"	// não é um caractere, e sim uma string
(''	abre parênteses
'\"'	aspa simples
'\\'	barra invertida
'\n'	nova linha
'\t'	tabulação
'\0'	nulo

Int

- Representa os números inteiros, positivos e negativos.
- Valor pode ser definido em decimal, octal com prefixo **0** ou hexadecimal com prefixo **0x**.
- Pode conter um sufixo **U** ou **u** para indicar sem sinal, ou **L** ou **l** para longo.
- Ocupa geralmente 4 bytes.

Exemplo de Valores Inteiros

-85	decimal
97u	decimal sem sinal
100L	decimal longo
012	octal = 10 em decimal
0x4b	hexadecimal = 75 em decimal

Float

- Representa os números que contém casas decimais, com precisão de 6 dígitos.
- Valor com casa decimal definida por ponto seguido de um sufixo **F** ou **f**.
- Ocupa sempre 4 bytes.

Double

- Também representa números com casas decimais, mas com precisão de 16 dígitos.
- Valor com casa decimal definida por ponto.
- Pode conter um sufixo **E** ou **e** para indicar a potência de 10, ou sufixo **L** ou **l** para longo.
- Ocupa geralmente 8 bytes.

Exemplo de Valores Ponto Flutuante

15.75f	float	
15.75	double	
15.75L	long double	
1.575E1	$1.575 \times 10^1 = 15.75$	double
1575E-2	$1575 \times 10^{-2} = 15.75$	double

Void

- Representação de vazio ou ausência de valor.
- Não tem tamanho definido e não ocupa espaço na memória.

Obs.: Ponteiros para void (void *) em C armazenam endereços de memória e o tamanho que ocupam depende da arquitetura da CPU (4 bytes em CPUs de 32 bits e 8 bytes em CPUs de 64 bits).

Modificadores de Tipos Primitivos

- Alteram o significado de um tipo, adaptando-o para uma situação específica (uma faixa de valores).
- Podem ser aplicados aos tipos primitivos do C (exceto void e float).
- Exemplo: um número pode ou não ter sinal, um número pode ampliar ou reduzir a faixa de valores.

Modificadores de Tipos no C

unsigned	sem sinal
signed	com sinal (padrão)
short	domínio reduzido
long	domínio estendido

Obs.: Em CPUs de 32 bits, long geralmente armazena 32 bits. Em CPUs de 64 bits, o long pode armazenar 32 ou 64 bits. Para garantir o mínimo de 64 bits utiliza-se o modificador long long.

Tamanho/Faixa de Valores dos Tipos

Tipo	Tamanho	Valor mínimo e máximo
(signed) char	1 byte	-128 a 127
unsigned char	1 byte	0 a 255
(signed) short int	2 bytes	-32768 a 32767
unsigned short int	2 bytes	0 a 65535
(signed) int	4 bytes	-2147483648 a 2147483647
unsigned int	4 bytes	0 a 4294967295
(signed) long int (signed) long long int	8 bytes	-9223372036854775808 a 9223372036854775807
unsigned long int unsigned long long int	8 bytes	0 a 18446744073709551615
float	4 bytes	-3.4E+38 a 3.4E+38 (6 casas)
double	8 bytes	-1.8E+308 a 1.8E+308 (15 casas)
long double	16 bytes	-1.2E+4932 a 1.2E+4932 (18 casas)

Definindo um Nome para um Tipo

- C permite criar novos nomes para os tipos.
- Ajuda na portabilidade do código.
- Forma de declarar:

typedef <tipo> <novo_nome>;

Exemplo de typedef

```
typedef unsigned char byte;
```

```
typedef unsigned short int u_short_int;
```

```
typedef long int bigint;
```

```
typedef float real;
```

Variáveis

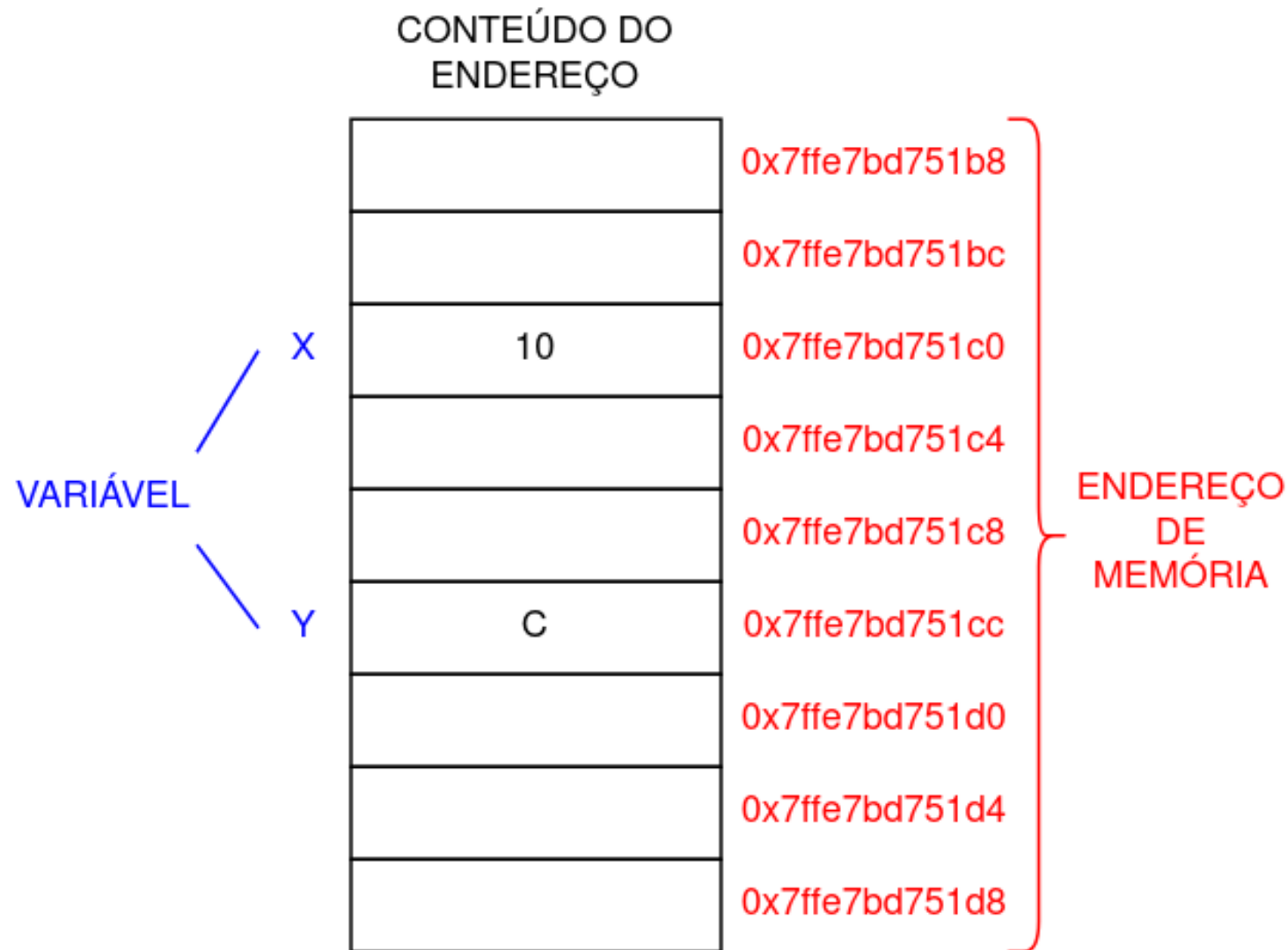
- Uma posição de memória nomeada que armazena um valor de acordo com o tipo definido.

- Forma de declarar:

<tipo> <nome_da_variável>;

- Deve-se iniciar com letra ou _, depois letras e dígitos.
- Nome preferencialmente em minúsculo.
- Não usar palavras reservadas.
- Pode-se atribuir um valor inicial à variável.

Alocando uma Variável na Memória



Exemplo de Variáveis

int var1, _var; // Nomes válidos

int 1var, @var; // Nomes inválidos

double valor_produto; // Nome no Padrão ANSI C

double valorProduto; // Nome no Padrão Camel-Case

int var1 = 0; // Declaração com valor inicial

int var1 = var2 = 0; // Declaração inválida

Escopo de Variáveis

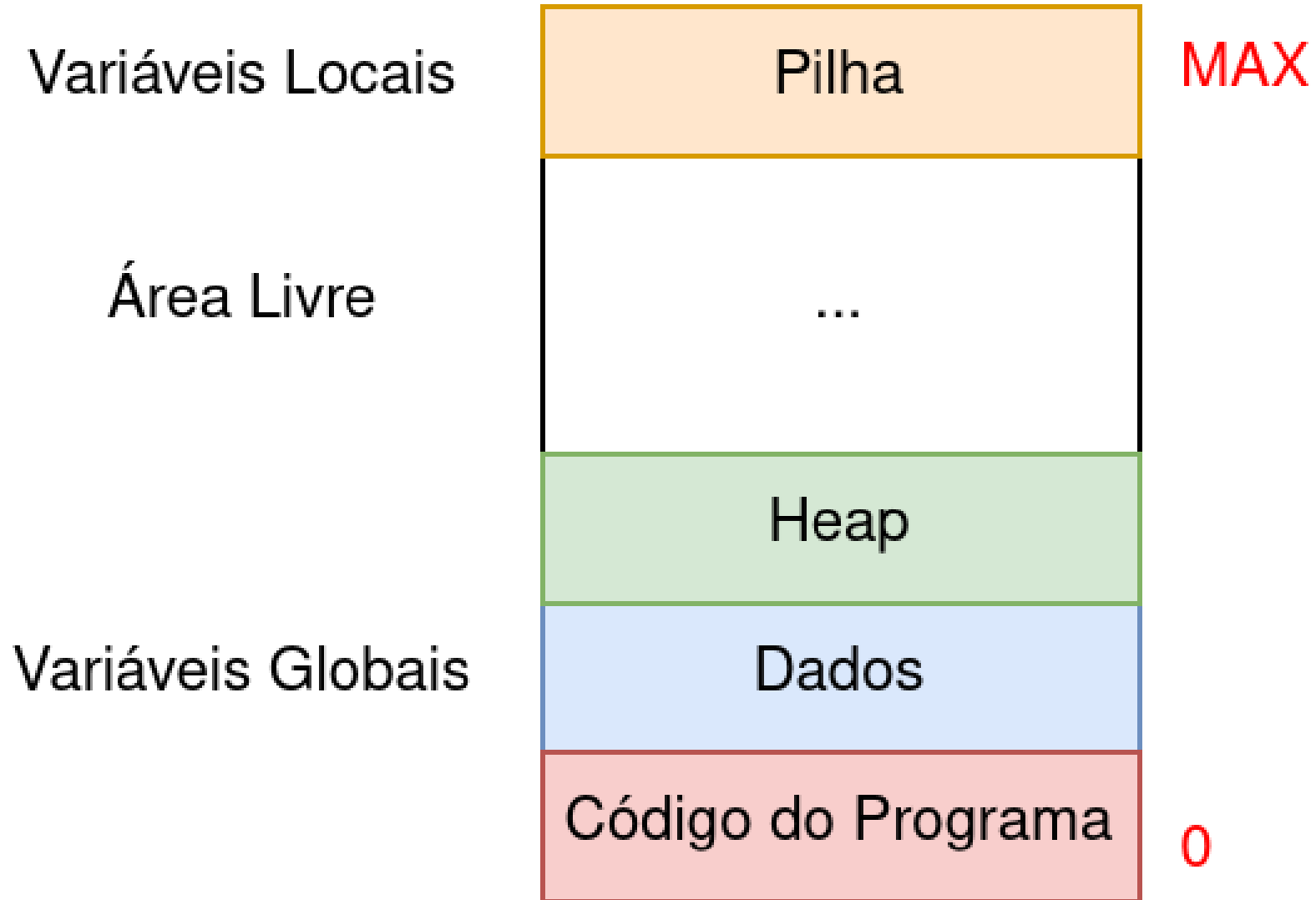
- Local

- Variáveis declaradas dentro de bloco { }.
- São reconhecidas somente dentro do bloco.

- Global

- Variáveis declaradas fora de bloco { }.
- São reconhecidas pelo programa inteiro.
- São acessíveis em qualquer parte do código.

Mapa Conceitual da Memória



Exemplo de Escopo de Variáveis

```
int count; // global
```

```
int main()  
{  
    count = 10;  
    int temp = count; // local  
  
    return 0;  
}
```

Constantes

- Variáveis com valores fixos.
- Formas de declarar:

#define <nome_da_constante> <valor>

const <tipo> <nome_da_constante> = <valor>;

- Deve-se iniciar com letra ou _, depois letras e dígitos.
- Nome preferencialmente em maiúsculo.
- Utiliza-se sublinhado _ como separador.

Exemplo de Constantes

// Pré-compilador C troca no código o nome pelo valor

```
#define SEXO_MASCULINO 1
```

```
#define MENSAGEM "isso é uma string"
```

```
#define PI 3.14
```

// Uma variável é alocada na memória com o valor fixo

```
const int SEXO_MASCULINO = 1;
```

```
const char *MENSAGEM = "isso é uma string";
```

```
const double PI = 3.14;
```

PI = 3.1416; // Não se pode alterar

Mapa da aula

Aula (teoria): Aula03–04 — Tipos e variáveis

Laboratório guiado: Pratica03

Atividade avaliativa: Atividade01

Sugestão de ritmo:

1) Conceitos nos slides 2) Prática no lab 3) Atividade sem 'receita'

Para estudar no livro

Livro-base: ANTONELLO, Ricardo. Linguagem C. 1. ed.

Leitura indicada: Cap. 4.1–4.6 e 4.11–4.12 (p. 28–32 e 40–41)

Exercícios do livro: Exercícios 4.13 (tipos)

Revise o capítulo após a aula e antes da prática.

Desafio (8 min)

Declare variáveis para idade, altura, inicial do nome e aprovado (0/1). Imprima com printf.

Trabalhe em dupla (instrução por pares).

Ao final, uma dupla compartilha a solução com a turma.

Quiz rápido (3 perguntas)

- 1) Qual a diferença entre 'a' e "a"?
- 2) Para que servem const e #define?
- 3) O que é cast?

Respostas no quadro / Mentimeter / Kahoot.

Referências

SCHILDT, Herbert. C: Completo e Total. 3. ed. Pearson Makron Books, 1997.

ANTONELLO, Ricardo. Linguagem C. 1. ed. Livro Digital, 2020.

ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene. Fundamentos da Programação de Computadores. 3. ed. Pearson, 2012.